

Práctica de laboratorio 1: Generación de datos sintéticos y almacenamiento de información

Autores: Iñaki Gutiérrez-Mantilla López y Lucas Miranda Lopez

1. Creación de datos sintéticos.

Primeramente se nos pide que generemos datos con la biblioteca *Faker*, para ello primeramente tenemos que proveer con ciertos datos fidedignos a la función que usará nuestra instanciación de *Faker*. Esta carga se hace una única vez al arrancar, en la función *cargar_datos_base()* del módulo *generador.py*, y produce tres estructuras:

- **Catálogo de vehículos.** Se lee *vehicles.json* con el módulo estándar *json* y se aplanan en una lista de diccionarios con fabricante, modelo, categoría y periodos de producción. El fichero contiene 23 fabricantes y 468 modelos repartidos en 10 categorías. Cinco modelos tienen más de un periodo de producción, así que guardamos todos los periodos en una lista en lugar de quedarnos solo con el primero, como hacíamos en el prototipo inicial. El valor 9999, que indica que el modelo sigue fabricándose, se sustituye por 2026.
- **Códigos postales y municipios.** Se lee el CSV del INE indicado en el enunciado (14.608 pares código postal–municipio) con *csv.DictReader*. El código postal se lee como texto para conservar los ceros iniciales (01001). Descartamos las 3 filas cuyo código empieza por «00», porque ese prefijo no corresponde a ninguna provincia (este era uno de los problemas que hacía fallar nuestra generación), quedan 14.605 filas.
- **Diccionario de provincias.** Asocia los dos primeros dígitos del código postal con la provincia (01-Álava, ..., 52-Melilla), según el anexo de Wikipedia que cita el enunciado. Al ser un diccionario, obtener la provincia de un código postal es una consulta en tiempo constante.

1.1. Conjunto de usuarios

Cada usuario es un diccionario con los campos que pide el enunciado.

La elección de la fila del CSV es uniforme y no está ponderada por población, de modo que un municipio pequeño aparece con la misma frecuencia que Madrid. Para generar datos de prueba nos parece suficiente; si se quisiera una distribución geográfica realista, bastaría con pasar los pesos de población a *random.choices*.

1.2. Conjunto de vehículos

Los vehículos se generan a partir de cada usuario, justo después de crearlo, y guardan su DNI en el campo *usuario_dni*, que es la referencia entre los dos conjuntos.

El número de vehículos sigue una Poisson con $\lambda = 0,9$ truncada a $[0, 4]$. Calculamos la función de masa para $k = 0, \dots, 4$ y usamos esos valores como pesos en *random.choices*. Como *random.choices* normaliza los pesos, el resultado es exactamente la Poisson condicionada a $k \leq 4$.

Cada vehículo cuenta con estos datos:

- **Fabricante, modelo y categoría:** se elige un modelo del catálogo de forma uniforme y de él salen los tres campos. Si el modelo tiene varios periodos de producción, se elige uno al azar.
- **Año:** entero uniforme dentro del periodo de producción elegido, de modo que nunca aparece un vehículo fabricado fuera de su periodo.
- **Matrícula:** depende del año. Hasta 1999 se usa el formato provincial histórico (M-1234-AB): sigla de una provincia, cuatro dígitos y dos letras. Desde 2000 se usa el formato nacional (1234-BCD): cuatro dígitos y tres consonantes, sin vocales, Ñ ni Q, como en las matrículas reales.
- **Bastidor:** 17 caracteres entre dígitos y letras mayúsculas, excluyendo I, O y Q, que la norma ISO 3779 prohíbe en el VIN para que no se confundan con 1 y 0. No calculamos el dígito de control ni la estructura WMI/VDS/VIS del VIN real, porque el enunciado solo pide una secuencia de 17 letras y números.
- **usuario_dni:** el DNI del propietario.

Como el modelo se elige de forma uniforme, la frecuencia de cada categoría refleja cuántos modelos tiene en el catálogo. Por ejemplo, SUV es la más frecuente (21,6 % de los vehículos) porque tiene 101 de los 468 modelos. Alrededor del 27 % de los vehículos son anteriores a 2000 y llevan matrícula histórica.

1.3. Provider propio de Faker

Faker no tiene métodos para generar un DNI con letra válida, matrículas españolas según el año ni números de bastidor, así que los implementamos en un Provider propio, *ProveedorEspana* (archivo *proveedor.py*), que hereda de *BaseProvider* y se registra con *fake.add_provider(ProveedorEspana)*. Dentro del Provider usamos los métodos de *BaseProvider* (*random_int*, *random_element*, *numerify*, *lexify*) en lugar del módulo *random*, para que los valores salgan del generador aleatorio de la propia instancia de Faker.

El DNI y la matrícula son claves primarias en las bases de datos relacionales, así que no pueden repetirse. Para ello usamos el método *fake.unique*, que recuerda los valores ya devueltos y vuelve a generar si sale uno repetido.

En una primera versión el Provider tenía un único método *matricula(anio)* y lo llamábamos con *fake.unique.matricula(anio)*. Con 1.000 usuarios funcionaba y con 10.000 casi siempre, pero con 100.000 la carga en SQLite y PostgreSQL fallaba siempre con «UNIQUE constraint failed: vehiculos.matricula». Al revisar el código de Faker vimos que *unique* guarda los valores vistos por cada combinación de método y argumentos: al pasar el año como argumento, solo evitaba repetidos entre vehículos del mismo año, y con unos 89.000

vehículos aparecían unas 30 matrículas repetidas entre años distintos. Lo resolvimos separando el método en *matricula_historica()* y *matricula_actual()*, sin argumentos. Como los dos formatos no pueden coincidir nunca, que cada método no repita valores garantiza que no se repite ninguna matrícula en todo el conjunto. Es justo lo que se dice en las transparencias, *unique* no siempre es la mejor solución cuando el modelo se complica.

El bastidor no lo hacemos único: con 33 símbolos posibles en 17 posiciones hay unas $6,5 \cdot 10^{25}$ combinaciones, así que la probabilidad de que se repita es despreciable y nos ahorramos mantener otro conjunto en memoria.

1.4. Generación

Para no tener todo el conjunto en memoria, *generador_datos()* es una función generadora: en cada iteración crea un usuario con sus vehículos y lo entrega con *yield* como una tupla (*usuario*, *vehiculos*). Esto tiene tres ventajas:

- La memoria no depende del número de usuarios, salvo por los valores que *unique* tiene que recordar (DNI y matrículas).
- Cada estrategia de almacenamiento consume los datos como le conviene: registro a registro (CSV, JSON, Avro, SQLite) o agrupándolos en lotes (Parquet, PostgreSQL, MongoDB).
- Los dos conjuntos se recorren en una sola pasada y cada usuario llega junto con sus vehículos, que es exactamente lo que necesitan los modos anidados de JSON, Avro y MongoDB.

Es la solución que proponen las transparencias para generar más datos de los que caben en memoria, funciones generadoras y procesamiento por lotes. En la sección 5 se comprueba que el pico de memoria apenas cambia entre 1.000 y 100.000 usuarios.

2. Almacenamiento en ficheros.

Cada formato está implementado como una estrategia (sección 4) que recibe el generador y un directorio base, y escribe sus ficheros en *<directorio>/<formato>/. Ninguna acumula el conjunto completo: todas escriben registro a registro o por lotes de 10.000 usuarios.*

2.1. CSV y Parquet

En ambos formatos se genera un fichero por conjunto de datos (*usuarios* y *vehiculos*), y la relación se mantiene con la columna *usuario_dni* de vehículos, que hace referencia al campo *dni* de usuarios.

CSV: Usamos *csv.DictWriter* del módulo estándar, que escribe cada diccionario como una fila. La cabecera se toma de las claves del primer registro y cada fila se escribe en cuanto se genera. Todo es texto, el año se guarda como la cadena "2009" y un teléfono fijo nulo se escribe como una cadena vacía, por lo que al leerlo no se distingue un nulo de un texto vacío.

Parquet: Usamos *pyarrow* directamente, sin pasar por Pandas. Se abre un *pq.ParquetWriter* por fichero con un esquema explícito y, cada 10.000 usuarios, se construye una tabla Arrow con *pa.Table.from_pylist* y se añade al fichero. Cada lote queda

en el fichero como un *row group*. El esquema explícito es necesario, si se infiriera del primer lote y en él todos los teléfonos fijos fueran nulos (posible con lotes pequeños), la columna quedaría con tipo *null* y el siguiente lote fallaría. Además, nos permite guardar el año como *int32*.

2.2. JSON Lines y Avro

Para estos formatos el enunciado pide dos modos: un único fichero jerárquico, en el que cada usuario lleva dentro la lista de sus vehículos, y un fichero por conjunto enlazado. Las dos estrategias escriben ambos modos en la misma pasada del generador, con tres ficheros abiertos a la vez: *datos_anidados*, *usuarios* y *vehiculos*.

¿Por qué JSON Lines? En lugar de un único array JSON usamos JSON Lines, un objeto JSON completo por línea. Con un array hay que gestionar los corchetes y las comas entre elementos y, para leerlo, *json.load* y la mayoría de librerías cargan el documento entero en memoria. Con JSONL cada línea es un JSON válido e independiente, así que se puede escribir registro a registro, leer línea a línea, añadir registros al final sin reescribir el fichero y partirlo en trozos para su procesamiento en paralelo. Escribimos con *ensure_ascii=False* para que las tildes y la ñ se guarden tal cual.

En el modo anidado los vehículos conservan el campo *usuario_dni*, que ahí es redundante porque el vehículo ya está dentro de su usuario. Lo mantuvimos por simplicidad, ya que el mismo diccionario sirve para los dos modos, aunque eliminarlo ahorraría algo de espacio.

Avro. Avro necesita un esquema, así que definimos tres con *fastavro*: *Usuario*, *Vehiculo* y *UsuarioConVehiculos*, que reutiliza los campos del usuario y añade *vehiculos* como un array de registros *Vehiculo*. El teléfono fijo se declara como la unión *["null", "string"]*, que es como se expresa un campo opcional en Avro, y el año como *int*. Para escribir usamos la clase *fastavro.write.Writer*, que permite añadir registros de uno en uno (fastavro los agrupa internamente en bloques), en vez de la función *fastavro.writer*, que recibe un único iterable: como escribimos tres ficheros a partir de la misma pasada, necesitamos escribir en cada uno por separado. El esquema se guarda en la cabecera del fichero, de modo que cualquier lector puede interpretarlo sin información adicional.

2.3. Comparativa de formatos

¿Usaríamos cualquier formato de forma indistinta?

No. Todos sirven para guardar estos datos, pero cada uno encaja mejor en un caso de uso:

- **CSV:** intercambio de datos tabulares sencillos con hojas de cálculo u otras herramientas, y carga masiva en un SGBD. Es universal y legible, pero no tiene tipos, los nulos son ambiguos y no admite anidamiento.
- **JSONL:** APIs web, ficheros de configuración, logs y datos semiestructurados o anidados, como los documentos de MongoDB. Es legible y flexible, pero es el que más ocupa. JSONL es preferible a un array JSON cuando hay muchos registros.
- **Parquet:** almacenamiento analítico y consultas que leen pocas columnas de muchos registros. Es el más compacto, pero no está pensado para escribir o actualizar registros sueltos.

- **Avro:** serialización de registros completos en sistemas de mensajería como Kafka o en procesos de ingesta, sobre todo cuando el esquema cambia a lo largo del tiempo. Es compacto y rápido de escribir, pero no es legible y para leer una sola columna hay que recorrer el registro entero.

Formatos basados en filas frente a formatos basados en columnas

En un formato por filas (CSV, JSON, Avro) los campos de un mismo registro están seguidos, así que añadir registros o leerlos completos es eficiente, pero para consultar una sola columna hay que leerlas todas. En un formato por columnas (Parquet) los valores de cada columna se guardan juntos, por lo que se puede leer solo la columna que interesa y se comprimen mucho mejor. A cambio, escribir exige acumular lotes de filas y reconstruir registros completos es más costoso, por eso es el formato típico de la analítica.

Ficheros separados frente a estructura jerárquica

Ficheros separados: cada instancia se guarda una sola vez, cada conjunto se puede consultar o actualizar por separado y se corresponde directamente con tablas relacionales. Como inconveniente, hay que hacer un *join* para reconstruir la información y el formato no garantiza la integridad referencial, nada impide que aparezca un *usuario_dni* que no existe.

Estructura jerárquica: leer un usuario con todos sus datos no necesita *joins* y la relación queda implícita en la estructura, lo que encaja con documentos (MongoDB) y APIs. Como inconveniente, consultar los vehículos por separado obliga a recorrer todos los usuarios, actualizar un vehículo implica reescribir el documento y, si la relación fuera de muchos a muchos (un vehículo con varios propietarios), los vehículos se duplicarían.

3. Almacenamiento en SGBD.

Los tres SGBD se implementan también como estrategias. En SQLite y PostgreSQL usamos el modelo relacional, con una tabla por conjunto y una *foreign key*, y en MongoDB el modelo documental con los vehículos embebidos. En todos los casos las tablas o colecciones se recrean al empezar, para que ejecuciones sucesivas no choquen con las claves primarias.

3.1. SQLite3

Toda la base de datos es un único fichero, *<directorio>/sqlite/datos.db*, sin servidor ni configuración. Creamos dos tablas, *usuarios*, con *dni* como clave primaria, y *vehiculos*, con *matricula* como clave primaria y *usuario_dni* como clave foránea hacia *usuarios*.

Cada usuario se inserta con *execute* y sus vehículos con *executemany*, todo en una única transacción que se confirma al final, ya que confirmar fila a fila sería mucho más lento. Hay que tener en cuenta que SQLite tiene tipado dinámico y no comprueba las claves foráneas salvo que se active *PRAGMA foreign_keys = ON*. En nuestro caso la integridad la garantiza la propia generación, ya que cada vehículo se crea a partir de un usuario existente.

3.2. PostgreSQL

Nos conectamos con *psycopg2* mediante una URI con usuario, contraseña, host, puerto y base de datos. A diferencia de SQLite, aprovechamos el tipado estricto (*VARCHAR(10)* para el DNI, *CHAR(5)* para el código postal, *SMALLINT* para el año y *NOT NULL* en los campos obligatorios) y la clave foránea sí se comprueba en cada inserción.

Insertamos por lotes de 10.000 usuarios con *execute_values*, que agrupa muchas filas en cada sentencia *INSERT* para reducir los viajes al servidor. En cada lote se insertan primero los usuarios y después sus vehículos para respetar la clave foránea, y toda la carga es una única transacción, así que si algo falla se deshace por completo.

3.3. MongoDB

Con *pymongo* nos conectamos también mediante una URI. Guardamos una colección *usuarios* en la que cada documento es un usuario con sus vehículos embebidos, es decir, el modo jerárquico, y usamos el DNI como *_id* para que MongoDB garantice que no se repite. Los documentos se insertan con *insert_many* en lotes de 10.000 y al final se crea un índice sobre *vehiculos.matricula* para poder buscar un vehículo por su matrícula.

Embedimos los vehículos porque la relación es 1:N con muy pocos elementos (como mucho 4) y la consulta más natural es un usuario con sus vehículos, que así se resuelve con una sola lectura. Si los vehículos se consultaran sobre todo por separado, sería mejor una colección aparte con el DNI como referencia.

Respecto a la pregunta de las transparencias sobre qué pasa con los datos tras un *insert*, *insert_many* añade el campo *_id* a los diccionarios que no lo tienen. En nuestro caso no afecta, porque cada documento es un diccionario nuevo que ya incluye su *_id*.

3.4. Comparativa de SGBD

Tampoco los usaríamos de forma indistinta:

- **SQLite:** es una biblioteca embebida, sin servidor, que guarda toda la base de datos en un fichero. Es ideal cuando la base de datos acompaña a una sola aplicación (apps móviles o de escritorio, prototipos, pruebas o cachés locales), pero solo admite un escritor a la vez y no está pensada para acceso por red.
- **PostgreSQL:** es un SGBD relacional cliente-servidor con tipado estricto, integridad referencial completa, transacciones ACID y buena concurrencia. Es la opción para el backend de una aplicación multiusuario en la que importa la integridad de los datos y para consultas SQL complejas. Por contraparte requiere instalación y administración del servidor.
- **MongoDB:** es un SGBD documental cliente-servidor, con esquema flexible y escalado horizontal. Encaja con datos semiestructurados o cuyo esquema cambia a menudo y con datos que se leen juntos, como un usuario con sus vehículos. Es menos adecuado cuando hay muchas relaciones entre entidades, porque no tiene claves foráneas.

Para este caso, con dos entidades bien definidas y una relación clara entre ellas, PostgreSQL nos parece la opción más sólida para un sistema real. MongoDB tendría

sentido si siempre se consultara el usuario completo, y SQLite para trabajar con los datos en local.

4. Diseño del código y ejecución.

4.1. Estructura y patrones de diseño

La aplicación se ejecuta desde línea de comandos y está dividida en módulos: *main.py* (argumentos y orquestación), *generador.py* (datos de referencia y generador), *proveedor.py* (Provider de Faker) y el paquete *estrategias*, con un módulo por cada formato o SGBD.

Patrón Strategy: cada mecanismo de persistencia es una clase que hereda de la clase abstracta *Estrategia* e implementa un único método, *guardar(generator, ruta_base)*. *main.py* solo conoce esa interfaz, sin saber si está escribiendo un CSV o insertando en PostgreSQL.

Función fábrica: siguiendo la idea del patrón Factory Method, *obtener_estrategia(formato, uri)* elige qué clase instanciar a partir del nombre del formato, consultando un registro (*ESTRATEGIAS*) que asocia cada formato con su módulo y su clase. El módulo solo se importa cuando se pide, así que si no está instalado *psycopg2* o *pymongo* el resto de formatos sigue funcionando.

Con este diseño, añadir un formato o SGBD nuevo consiste en crear su módulo con una subclase de *Estrategia* y añadir una línea al registro, sin tocar *main.py* ni el generador. Cada fichero de código incluye una cabecera con el nombre de los autores.

4.2. Uso desde línea de comandos

main.py recibe el número de usuarios (*-n*), el formato o SGBD (*-f*, que admite csv, parquet, json, avro, sqlite, postgres o mongodb), el directorio de salida (*-o*, por defecto *datasets*) y, para PostgreSQL y MongoDB, la URI de conexión (*--uri*). Por ejemplo:

```
python main.py -n 10000 -f postgres --uri postgresql://user:pass@localhost:5432/bdia
```

Las dependencias externas son *faker*, *pyarrow*, *fastavro*, *psycopg2* y *pymongo*.